

研发场景十大热门 Skills 推荐

本文作者：JiaQi，TRAE 技术文档工程师

 本文面向实际研发场景，整理并推荐了一组可直接应用于日常开发流程的 Agent Skills。这些 Skills 覆盖前端设计、前后端开发、代码审查、自动化测试、CI/CD、问题修复以及文档维护等常见环节，分别针对具体任务提供明确的能力边界与使用场景说明，帮助你在不同阶段选择合适的 Skills，提高开发效率。

前端设计

名称	frontend-design
作者	Anthropic
地址	https://github.com/anthropics/skills/tree/main/skills/frontend-design
简介	该 Skill 旨在创建具有独特性和高设计品质的前端界面，能够达到生产级别 (production-grade) 的标准。它的核心目标是避免生成千篇一律、缺乏独特风格的“AI 风格”界面，而是通过在设计上有意地选择大胆、明确的美学方向（例如：极简、复古、未来感、野兽派等），并注重排版、色彩、动效、空间布局等细节，来打造出令人印象深刻、具有艺术感的前端页面。
资源文件	该 Skill 中仅包含一个 SKILL.md 文件。
应用场景	构建网页组件或页面 当你需要从零开始创建一个具体的 UI 元素时，例如一个 React 组件、一个 HTML/CSS 布局，或一个独立的静态页面。该 Skill 会确保这个组件不仅功能完善，而且在视觉上具有辨识度。 开发完整的 Web 应用或网站 当你需要构建一个完整的应用界面时，比如一个产品的 Landing Page、一个数据仪表盘或一个小型网站。该 Skill 会从整体出发，确立一个统一且鲜明的设计风格，并将其贯彻到应用的每一个角落。 美化或重塑现有界面 当你有一个已经存在的、但设计平庸的网页或应用，并希望提升其视觉品质时。该 Skill 会专注于美化层面的工作，通过引入独特的字体、创意的色彩方案、精致的动效和新颖的布局，来重塑界面的整体美感。

前端开发

名称	cache-components
作者	vercel
地址	https://github.com/vercel/next.js/tree/canary/.claude-plugin/plugins/cache-components/skills/cache-components
简介	该 Skill 旨在将 Next.js 的 Partial Prerendering (PPR) 和缓存组件（Cache Components）的最佳实践，通过 AI 助手无缝集成到开发工作流程中。当项目环境启用 <code>cacheComponents: true</code> 配置时，该 Skill 将被激活，为你提供自动化的代码生成与优化能力。
资源文件	除 SKILL.md 文件外，该 Skill 中还包含以下文件： PATTERNS.md：一份详细的说明文档，通过丰富的代码示例和场景解释，来展示如何高效、正确地使用 Cache Components。 REFERENCE.md：Cache Components 的官方 API 参考手册，它精确定义了所有相关函数、指令和核心概念，用于查找具体技术细节。 TROUBLESHOOTING.md：Cache Components 的故障排查指南，提供了一系列常见错误的解决方案、调试清单和实用技巧，用于诊断并解决使用缓存组件时遇到的问题。
应用场景	自动生成缓存优化的数据组件 当创建数据获取组件时，系统会自动应用最优的渲染策略：针对可共享数据（如产品目录），使用 <code>'use cache'</code> 语法进行缓存；针对用户专属内容，则自动添加 <code><Suspense></code> 边界，以实现动态流式渲染。 自动实现数据变更后的缓存失效 当生成用于修改数据的 Server Action 时，系统会自动注入缓存失效逻辑（如 <code>updateTag()</code> 方法）。这能确保数据变更后，相关缓存会立即更新，从而保障整个应用的数据一致性。 智能化页面构建与代码现代化 在构建页面或审查代码时，系统会强制遵循 Partial Prerendering（PPR）架构规范，以实现最优的加载性能。同时，它能识别并给出现代化改造建议，例如用组件级的 <code>'use cache'</code> 替代已过时的页面级缓存配置。

全栈开发

名称	fullstack-developer
作者	Shubhamsaboo
地址	https://github.com/Shubhamsaboo/awesome-llm-apps/tree/main/awesome_agent_skills/fullstack-developer

简介	该 Skill 的主要作用是扮演一个精通现代 Web 开发技术的全栈专家角色。它专注于使用 JavaScript/TypeScript 技术栈，特别是 React (Next.js)、Node.js 和主流数据库，来帮助你完成各类 Web 开发任务。
资源文件	该 Skill 中仅包含一个 SKILL.md 文件。
应用场景	- 构建完整的 Web 应用：从前端到后端，提供完整的解决方案。 - 开发 API：创建 RESTful 或 GraphQL 风格的后端接口。 - 创建前端界面：使用 React 或 Next.js 构建现代化的用户界面。 - 数据库和数据建模：设计和设置如 PostgreSQL 或 MongoDB 等数据库。 - 实现用户认证与授权：集成 JWT、OAuth 等认证机制。 - 部署与扩展应用：提供在 Vercel、Netlify 等平台上的部署指导。 - 集成第三方服务：在应用中接入外部服务。

代码审查（前端）

名称	frontend-code-review
作者	langgenius
地址	https://github.com/langgenius/dify/tree/main/.agents/skills/frontend-code-review
简介	该 Skill 的核心功能是自动化审查前端代码（尤其针对 .tsx、.ts、.js 等文件）。它会依据预定义的规则清单，从代码质量、性能表现、业务逻辑等维度对代码开展全面分析。审查完成后，系统将生成结构清晰的报告，把发现的问题划分为“紧急待修复”和“改进建议”两类，并标注具体的代码位置、提供可落地的修复方案，助力你提升代码质量。
资源文件	除 SKILL.md 文件外，该 Skill 中还包含以下文件： references/business-logic.md：定义与特定业务场景相关的规则，以防止出现逻辑错误或 Bug。例如：规定在某些组件中不能使用 workflowStore，因为在特定使用场景下（如从模板创建管道时）它会导致页面白屏。 references/code-quality.md：包含一系列通用编码规范，旨在保持代码的整洁、一致和可维护性。例如：推荐使用 <code>cn</code> 这样的工具函数来处理动态的 CSS 类名，而不是手动拼接字符串，以保证代码风格统一。 references/performance.md：专注于前端性能，提供避免常见性能问题的最佳实践。例如：建议将传递给子组件的对象或数组等复杂属性用 <code>useMemo</code> 包裹起来，以防止不必要的组件重渲染。
应用场景	- 审查待提交的变更 在你准备提交代码（<code>git commit</code>）前，可使用该 Skill 审查所有已修改或已暂存的文件，提前识别并标记不符合规范的代码，避免潜在的 Bug 或性能问题被合入代码库。 - 审查指定的文件

	当你需对特定文件或模块进行重构、优化或问题排查时，可将相关文件交由该 Skill 审查，从而对目标文件开展针对性深度分析，快速获取该文件的质量评估结果及改进建议。 获取结构化的修复报告 代码审查发现问题后，该 Skill 会自动整理并输出标准化报告。这份报告不仅罗列问题，还会按紧急程度排序，同时标注问题对应的文件路径、行号、相关代码片段，并给出可执行的修复方案。
--	---

代码审查（通用）

名称	code-reviewer
作者	google-gemini
地址	https://github.com/google-gemini/gemini-cli/tree/main/.gemini/skills/code-reviewer
简介	该 Skill 旨在引导 AI 开展专业且全面的代码审查工作。它既支持审查本地代码改动（包括已暂存和未暂存的变更），也可审查远程代码合并请求（Pull Request，简称 PR）。审查的核心目标是保障代码的正确性、可维护性，并确保代码符合项目既定的规范标准。
资源文件	该 Skill 中仅包含一个 SKILL.md 文件。
应用场景	审查远程 PR 当你完成功能开发或问题修复并提交 PR 后，可发起 AI 审查请求。你只需提供 PR 编号或 URL（例如："Review PR #123"），AI 会自动检出（checkout）该 PR 的代码，运行项目预设的检查脚本（如 <code>npm run preflight</code>），同时阅读 PR 描述与评论以理解开发目标，随后对代码开展深度分析并给出反馈。 审查本地代码变更 若你希望在提交代码或创建 PR 前，先对本地修改进行审查，只需发出“审查我的代码”等类似指令即可，无需提供 PR 相关信息；AI 会通过 <code>git status</code>、<code>git diff</code> 等命令，检查工作区中已暂存（staged）和未暂存（unstaged）的代码改动，进而对这些变更进行分析并反馈。 提供深度分析与结构化反馈 无论是审查远程 PR 还是本地代码变更，AI 都会从多维度开展深度的代码质量分析，涵盖正确性、可维护性、可读性与执行效率、安全性与测试完整性等维度。最终，AI 会以结构化形式输出反馈，内容包括总体概述、具体发现（关键问题、改进建议）以及明确的结论（如批准合并或要求修改）。

网页应用测试

名称	wepapp-testing
----	----------------

作者	Anthropic
地址	https://github.com/anthropics/skills/tree/main/skills/webapp-testing
简介	该 Skill 是一个基于 Playwright 构建的本地 Web 应用测试工具集，支持前端功能验证、UI 行为调试、页面截图及浏览器控制台日志采集，适配「先侦查后行动」的测试流程。工具集提供示例脚本与辅助脚本，覆盖静态 HTML 自动化测试、元素定位、日志抓取能力，并可在多服务场景下统一管理服务器生命周期。
资源文件	除 SKILL.md 文件外，该 Skill 中还包含以下文件： • examples/console_logging.py：演示了在自动化测试时如何捕获并保存网页的控制台日志，用于调试和监控。 • examples/element_discovery.py：展示了如何自动发现和列出页面上所有的按钮、链接和输入框等可交互元素，是编写自动化脚本前的重要步骤。 • examples/static_html_automation.py：示范了如何直接对本地的静态 HTML 文件进行自动化操作（如点击、填表），而无需通过网络服务器。 • scripts/with_server.py：自动化辅助脚本。在执行一个主命令（例如自动化测试脚本）之前，先启动一个或多个依赖的服务（如前端或后端服务器），并确保这些服务完全就绪后，再执行主命令，最后在命令结束后自动关闭所有服务。
应用场景	• 自动验证前端功能 当你在本地开发 Web 应用（如使用 React、Vue、Svelte 等框架），并希望验证某个新功能是否正常运行时，只需用自然语言告知 AI 测试需求（例如：“帮我测试登录功能”），AI 会自动编写 Playwright 脚本来模拟用户操作，并向你反馈页面状态或内容是否符合预期。 • 调试与分析 UI 行为 若你发现页面中某元素渲染异常或交互行为异常，可向 AI 下达指令（例如：“截取首页完整截图”或“检查按钮的 DOM 结构”），AI 会执行对应脚本，捕获截图或获取 HTML 内容，并将侦察结果返回给你，助力快速定位问题。 • 处理需要后台服务的复杂交互 若你的应用为前后端分离架构，测试前端功能需后端 API 服务同步运行，只需告知 AI 项目结构与启动命令，AI 会借助 <code>with_server.py</code> 脚本来同时启动所有必需服务，再运行测试脚本，确保测试在完整的环境中执行。 • 测试静态 HTML 文件 若你有不依赖服务器的纯静态 HTML 页面，需验证其内容或结构，只需向 AI 提供文件路径与测试需求，AI 会编写脚本并通过 <code>file://</code> 协议在浏览器中打开该文件，完成验证操作。

CI/CD：PR 创建

名称	pr-creator
作者	google-gemini

地址	https://github.com/google-gemini/gemini-cli/tree/main/.gemini/skills/pr-creator
简介	该 Skill 的核心作用是引导并自动化创建高质量、符合规范的拉取请求（Pull Request，简称 PR）。它通过标准化工作流程，确保每一次代码提交均遵循项目预设的模板与质量检查标准，从而提升代码审查效率、保障团队协作的一致性。
资源文件	该 Skill 中仅包含一个 SKILL.md 文件。
应用场景	一键创建符合规范的 PR 当你在本地完成新功能开发或 Bug 修复，并已提交代码（git commit）后，可调用此 Skill，让 AI 自动执行分支检查、查找并应用 PR 模板、运行预检脚本（如测试和 linting），并最终生成一个标题和描述都完全符合项目规范的 PR。 引导贡献者完成首次代码提交 当新团队成员或外部贡献者不熟悉项目的提交流程和规范时，可以使用此 Skill，让 AI 以智能向导的形式，自动完成模板查找、脚本执行等繁琐操作，仅需用户填写必要的标题与描述，大幅降低代码贡献门槛。 自动执行创建 PR 前的质量检查 在正式创建 PR 之前，可以调用该 Skill，让 AI 自动运行项目预设的 preflight 脚本，执行所有必要的构建、单元测试和代码风格检查。如果任何检查失败，AI 会中止提交流程并提示开发者进行修复，节约了审查者的时间和精力。

Linting 和格式错误修复

名称	fix
作者	facebook
地址	https://github.com/facebook/react/tree/main/.claude/skills/fix
简介	这个 skill 的核心作用是自动化地修复代码格式并检查代码规范（linting）错误。它通过执行两个关键命令来保证代码质量： <code>yarn prettier</code>：自动格式化已修改的文件，统一代码风格。 <code>yarn linc</code>：检查代码中是否存在 linting 错误（这些是 Prettier 无法修复的，例如未使用的变量、逻辑错误等），这些错误通常会导致持续集成（CI）失败。 最终目标是确保代码在提交前符合项目规范，从而顺利通过 CI/CD 流程。
资源文件	该 Skill 中仅包含一个 SKILL.md 文件。
应用场景	提交代码前的预防性检查 在你完成编码，执行 <code>git commit</code> 之前，运行该 Skill，让 AI 自动清理代码格式，并提示任何需要手动修复的 linting 错误。 修复已发现的 linting 或格式问题

	当你在编码过程中或接手他人代码时，发现当前工作区内存在明显的格式混乱或 linting 错误提示（例如，IDE 的警告），可以立即运行该 Skill，快速解决当前已知的代码质量问题，从而在开发过程中保持代码的整洁和可读性。 • 解决持续集成（CI）失败问题 当一个提交被推送到服务器后，CI 流水线报告了因 linting 或格式错误导致的失败。此时你可以在本地对应的分支上运行此 Skill，让 AI 自动修复格式问题，并列出需要手动更正的 linting 错误，帮助你快速定位并解决问题，然后提交修复。
--	--

技术文档更新

名称	update-docs
作者	vercel
地址	https://github.com/vercel/next.js/tree/canary/.claude/skills/update-docs
简介	该 Skill 是一套用于更新 Next.js 项目文档的引导式工作流，核心作用是帮助你根据源代码的变更，来分析、更新和创建相关的文档，确保代码和文档保持同步。它特别为审查 Pull Request (PR) 时的文档完整性检查而设计，通过一系列标准化的步骤来规范文档的修改过程。
资源文件	除 SKILL.md 文件外，该 Skill 中还包含以下文件： • references/CODE-TO-DOCS-MAPPING.md：定义了源代码和文档之间的映射关系。简单来说，它就像一张地图，告诉 AI 当某个代码文件发生变化时，应该去更新哪个文档文件。 • references/DOC-CONVENTIONS.md：风格指南和规则手册，详细规定了文档的格式、结构和写作风格，以确保所有文档都保持一致性和高质量。
应用场景	• 分析代码变更对文档的影响 提交代码变更后，可以调用该 Skill 来分析哪些文档文件需要更新。 它会通过 <code>git diff</code> 命令检查你的分支与 <code>canary</code> 分支之间的差异，并根据预定义的映射关系 (<code>references/CODE-TO-DOCS-MAPPING.md</code>)，找出与变更的代码文件相对应的文档文件。 • 更新现有的文档 对于已经存在的文档，当其对应的功能或 API 发生变化时（例如组件新增了 props、函数行为变更），该 Skill 会引导你更新现有文档。 它会提示你如何添加或修改 props 表格、更新代码示例、添加废弃通知等，并遵循项目固有的文档规范（例如，使用 <code><AppOnly></code> / <code><PagesOnly></code> 来区分不同路由的内容）。 • 为新功能创建脚手架文档 当你在项目中添加了一个全新的功能时（例如一个新的组件、函数或配置项），该 Skill 可以帮你快速创建符合规范的新文档。 它为不同类型的文档（如 API 参考、指南）提供了标准模板，确保新文档的结构、命名和元信息（Frontmatter）都符合项目要求。

查找 Skill

名称	find-skills
作者	vercel
地址	https://github.com/vercel-labs/skills/tree/main/skills/find-skills
简介	该 Skill 主要作用帮助你发现并安装 Agent Skill。 它依托名为 <code>skills</code> 的命令行工具（CLI），让你可以从开放的 Agent Skill 生态中搜索、安装与管理各类模块化技能包；这些技能可扩展 Agent 能力，为其补充特定领域知识、标准化工作流与工具能力。
资源文件	该 Skill 中仅包含一个 SKILL.md 文件。
应用场景	探索未知的 Skill 当你希望 Agent 帮忙处理某个特定领域的任务，但不确定 Agent 是否具备相应能力时，可以使用此 Skill 进行探索。例如，当你询问“你能帮我评审代码吗？”或“如何为我的项目生成文档？”时，该 Skill 会被激活，主动在技能市场中搜索与“代码评审”或“文档生成”相关的能力，并将找到的可用技能呈现给你。 查找特定的 Skill 当你明确知道需要一个 Skill 来解决特定问题，但不知道具体是哪个 Skill 时，可以主动调用此 Skill 进行精确查找。例如，你可以直接说“帮我找一个用于 React 性能优化的 skill”，该 Skill 会将“React 性能优化”作为关键词进行搜索，并返回最匹配的技能选项，如“vercel-react-best-practices”。 提供可执行的 Skill 安装建议 当该 Skill 找到一个或多个匹配的 skill 后，它会自动整理并输出一份标准化的推荐信息。这份信息不仅包含技能的名称和功能简介，还会提供一键安装指令（<code>npx skills add ...</code>）以及指向技能详情页的官方链接。

欢迎大家安装尝试～先从会用 Skills 开始！

更多信息请关注TRAE企业版服务号👉



微信搜一搜



TRAE企业版